

TD 1 : Algorithme Parallèle

Ex. 1: Écrire un algo parallèle qui calcule la somme de 2 tableaux A et B de taille n dans un tableau C.

-> Mémoire partagée

-> Autant de mémoire que nécessaire.

```
C = malloc(sizeof(int)*n);
```

```
Pour i = 0 à n-1 faire en parallèle*:
```

```
    C[i] <- A[i] + B[i]
```

```
finpour
```

Justification* : Les calculs sont distincts. D'une itération à une autre, on n'utilise pas de variable commune ou liée. Conditions de **Bernstein.

On a n unités de calculs

$P = n$

$T_1(n) = O(n)$

$T_p(n) = 1$

$S_p(n) = T_1/TP = n/1 = n$

$E_p(n) = SP(n) / P = n/n = 1$

$E_p(n) = \frac{T_1(n)}{W_p(n)} = \frac{S_p(n)}{p}$	$S_p(n) = \frac{T_1(n)}{T_p(n)}$	$W_p(n) = p \cdot T_p(n)$
---	----------------------------------	---------------------------

- Ok maintenant imaginons qu'on n'ait pas autant de threads qu'on veut, on ne peut qu'utiliser **p unités**, avec $p < n$.

Répartition des données (itérations) par blocs

Modifier l'algorithme pour utiliser P unités avec $P < n$ et une répartition par blocs

```
reste <- n%p
```

```
pour u allant de 0 à P-1 faire en parallèle:
```

```
    nbCases[u] <- floor(n/P) // Qté min de travail de toutes les unités**
```

```
    si u < reste alors // on traite les reste 1ères unités
```

```
        nbCases[u]++ // on ajoute 1
```

```
        deb[u] <- u*nbCases[u]
```

```
    sinon
```

```
        deb[u] <- u*nbCases[u] + reste
```

```
    fini
```

```
    pour i[u] de deb[u] à deb[u] + nbCases[u] - 1 faire: [Séquentiel ! partie locale]
```

```
        C[i[u]] <- A[i[u]] + B[i[u]]
```

```
    finpour
```

$T_p(n) = \lceil n/P \rceil$
 $S_p(n) = n / \lceil n/P \rceil \approx P$
 $E_p(n) \approx 1$

****ATTENTION : On met nbCases[u] pour avoir des variables nbCases locales à chaque unité. Sinon tout le monde essaie de modifier nbCases et le comportement devient non-déterministe.**

Modifier l'algorithme pour utiliser une répartition cyclique (comme si on donnait des cartes à chaque thread)

| -012012012012010201201201021021020120120120120-|
 | -__-|

****Niggolas's code**** *Soit j'ai cook soit je suis cooked*

```

div <- N/P
Pour u de 0 à div en parallèle
    si u*p >= N
        return
    C[u*p] = A[u*p] + B[u*p]
finpour
**UI**
  
```

Correction:

```

Pour u de 0 à P-1 faire en parallèle:
    i[u] <- u
    tant que u i[u] < n faire:
        C[i[u]] <- A[i[u]] + B[i[u]]
        i[u] += p
    fintantque
finpour
  
```

[NOTE] La version cyclique n'est pas idéale sur CPU car elle ne garantit pas une bonne utilisation du cache contrairement à la version par blocs. Par contre ça peut être bien plus intéressant sur GPU.

Exercice 2 : Réduction

Définition : Réduire plusieurs valeurs à une seule



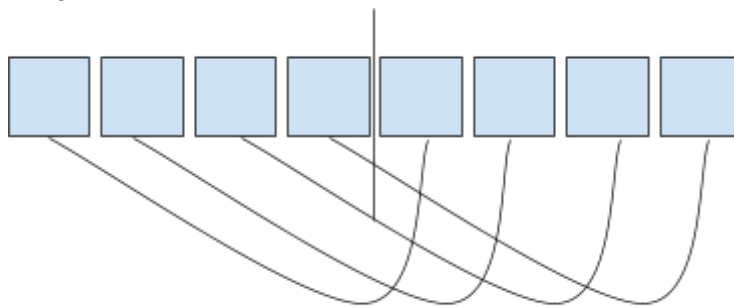
-> ne pas modifier le tableau T initial -> copie ($\log_2(n)$)

-> unités illimitées

*On peut d'une part faire deux par deux, et stocker le résultat sur le membre de gauche
Ou on peut prendre le premier et le dernier, puis le deuxième et l'avant-dernier, et ainsi de suite. L'avantage est que tous les résultats sont adjacents, à la moitié gauche. Puis on refait le processus jusqu'à avoir une seule case.*

Problème : Accès mémoire. On va au début, puis à la fin, puis au début, etc. On parcourt une grosse distance, et d'autres parcourent une petite. Il faut un accès régulier.

A la place on va faire ça :



Cela fonctionne si on a un tableau pair. Si c'est impair, il faut le garder pour l'étape suivante pour l'additionner à la fin sûrement

Algo :

pour i allant de 0 à n-1 *faire en parallèle:*

tmp[i] <- T[i] // On fait la copie en parallèle comme des bonhommes

finpour

nb <- n

tant que nb > 1 *faire en séquentiel* : //étapes de réduction

r <- nb%2 //élément central éventuel

nb <- floor(nb/2) // 1re moitié du tableau, on exclut l'élément central s'il y en a un
(floor est équivalent à une division entière, efficace)

pour u allant de 0 à nb-1 *faire en parallèle:*

tmp[u] += tmp[u+nb+r]

nb += r // Mise à jour pour étape suivante

finpour

fintantque

retour tmp[0]

p = O(n)

Tn(n) = log₂(n)

Sn(n) = n/log₂(n)

En(n) = 1/log₂(n)

Coût mémoire : $Mn(n) = \Theta(n)$

Principe de Brent (nombre d'unités optimal sur un algo existant) : $E = S/P \Rightarrow E = 1 \Rightarrow P = S/E = S$

Réécrire l'algorithme adapté à S unités ($=n/\log_2(n)$).

-> chaque unité calcule la somme partielle d'un bout du tableau -> $\log_2(n)$ données à traiter par unité.

-> répartition par blocs (moins simple à écrire) ou cyclique (plus simple)

[C'est bien pour l'utilisation mémoire aussi, car on a besoin de stocker uniquement S variable (1 par somme partielle, donc 1 par unité) et donc pas besoin de copier tout le tableau.]

$P \leftarrow n/\log(n)$

pour u allant de 0 à P-1 *faire en parallèle*:

$i[u] \leftarrow u$

$s[u] \leftarrow 0$

 tant que $i[u] < n$ faire:

$s[u] += T[i[u]]$

$i[u] += P$

 fintantque

finpour

rés $\leftarrow$ réductionSimple(s,P)

$Tp(n) = \log_2(n) + \log_2(n/\log_2(n)) =$ [2ème partie à cause de la réduction] $= O(\log_2(n))$

$Mp(n) = \Theta(n/\log_2(n))$

Double avantage : On économise de la mémoire, et on a un temps pareil, avec moins d'unités, donc plus d'efficacité.

Ex 3: Algo qui calcule les sommes par lignes d'un tableau 2D de taille $m \times n$ dans sL

ligne(T,l)

col(T,L)

pour l de 0 à N-1 *faire en parallèle (*m)*

$sL[l] \leftarrow B(+, \text{ligne}(\text{tab}, l), n)$ ($n/\log_2(n)$)

finpour

$T1(m,n) = m \times n$

$P = m \times n / \log_2(n)$

$Tp(m,n) = \log_2(n)$

$Sp(m,n) = m \times n / \log_2(n)$

$Ep(m,n) = Sp/p = 1$

Ajout du calcul des colonnes aussi, pas que des lignes

```

pour g de 0 à 1 faire en parallèle (2 gr)
  si g == 0 alors
    pour l de 0 à m-1 faire en parallèle (m)
      sL[l] <- B(+,ligne(tab,l), n) (n/log2(n))
    finpour
  sinon
    pour c de 0 à N-1 faire en parallèle (n)
      sC[c] <- B(+,col(tab,c), m) (m/log2(m))
    finpour
finpour

```

(Une partie fait les lignes, l'autre les colonnes)

$P = mn/\log(n) + mn/\log(m) = mn(\log(m)+\log(n))/\log(n)\log(m)$
 $Tp(m,n) = \max(\log(n), \log(m))$ (vu que c'est en parallèle, on prend le plus grand)
 $Sp(m,n) = 2*m*n / \max(\log(n), \log(m))$
 $Ep(m,n) = Sp / p = 2*\log(n)\log(m) / \log(m)+\log(m)*\max(...)$
 $= 2*\min(\log(n),\log(m)) / \log(m)+\log(m)$
 $= 1$ si $m = n$ et < 1 sinon

Variante :

```

pour u de 0 à m+n-1 faire en parallèle
  si u < m alors
    sL[u] <- B(+,...)
  sinon
    c <- u-m
    sC[c] <- B(+,...)

```

Exercice 4 : Traitement d'une image 1000*1000 niveaux de gris 1 pixel = 1 byte

Etapes :

- Lecture : 1 octet à la fois
- Réduction : 100*100 avec coût par pixel = 2000
- Intégration : 8 opération/pixel entiers
- Sauvegarde : 1 octet à la fois

	1	2	3	4
Degré	1	100*100	100*100	1
Coût élémentaire	1000*1000	2000	8	4*100*100

4 car on passe d'**octet** à **entier**

$T1 = 1 * 1000*1000 + 100*100 * 2000 + 8 * 100*100 + 4 * 100 * 100 = 21\ 120\ 000$
 T_{inf} (temps parallèle minimum) = Somme des coûts élémentaires = 1 042 008

Sinf $\approx$ 20,27

$T_{10} = \text{arrondisup}(1/10) * 1000 * 1000 + \text{arrondisup}(100 * 100 / 10) * 2000 + 8 * \text{arrondisup}(100 * 100 / 10) + \text{arrondisup}(1/10) * 4 * 100 * 100 = 3048000$

$S_{10} = 6,93$

$E_{10} = 0,69$

Trouver le plus grand P possible pour avoir $E_p \geq .5$

$E_p = 1 / (1 + (p-1)R_s)$

R_s = ratio de séquentialité dans le temps total

$= (10^6 + 4 * 10^4) / (21\ 120\ 000) = 0.05$

On veut du 0.5, pas du 0.05 🤖

$\frac{1}{2} = 1 / (1 + (P-1)R_s)$

$p = 1 / R_s + 1 \approx 21$

$E_{21} = 0,503$

$E_{22} = 0,491$

Parallélisme de tâches

On doit calculer n valeurs de la fonction $f(x) = f_1(x) * f_2(x) + f_3(x)$

avec les coûts **10 20 30**

-> Les accès arithmétiques et mémoire coûtent 1

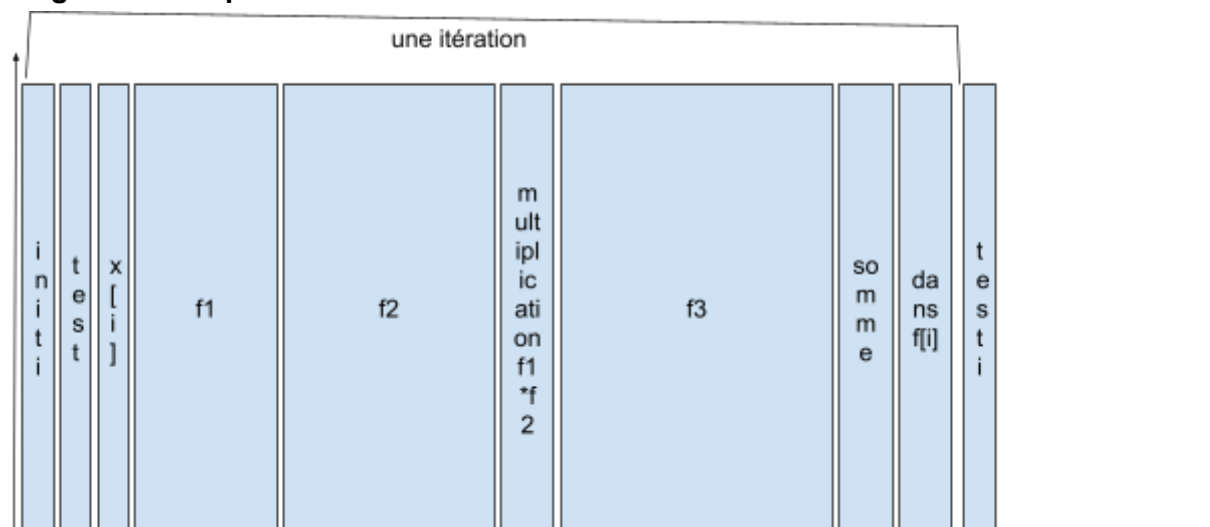
Algo séquentiel:

pour i de 0 à N-1 faire

$f[i] = f_1(x[i]) * f_2(x[i]) + f_3(x[i])$

finpour

Diagramme temporel :



Une itération = **66**

$T_1(n) = 1 + 66n$

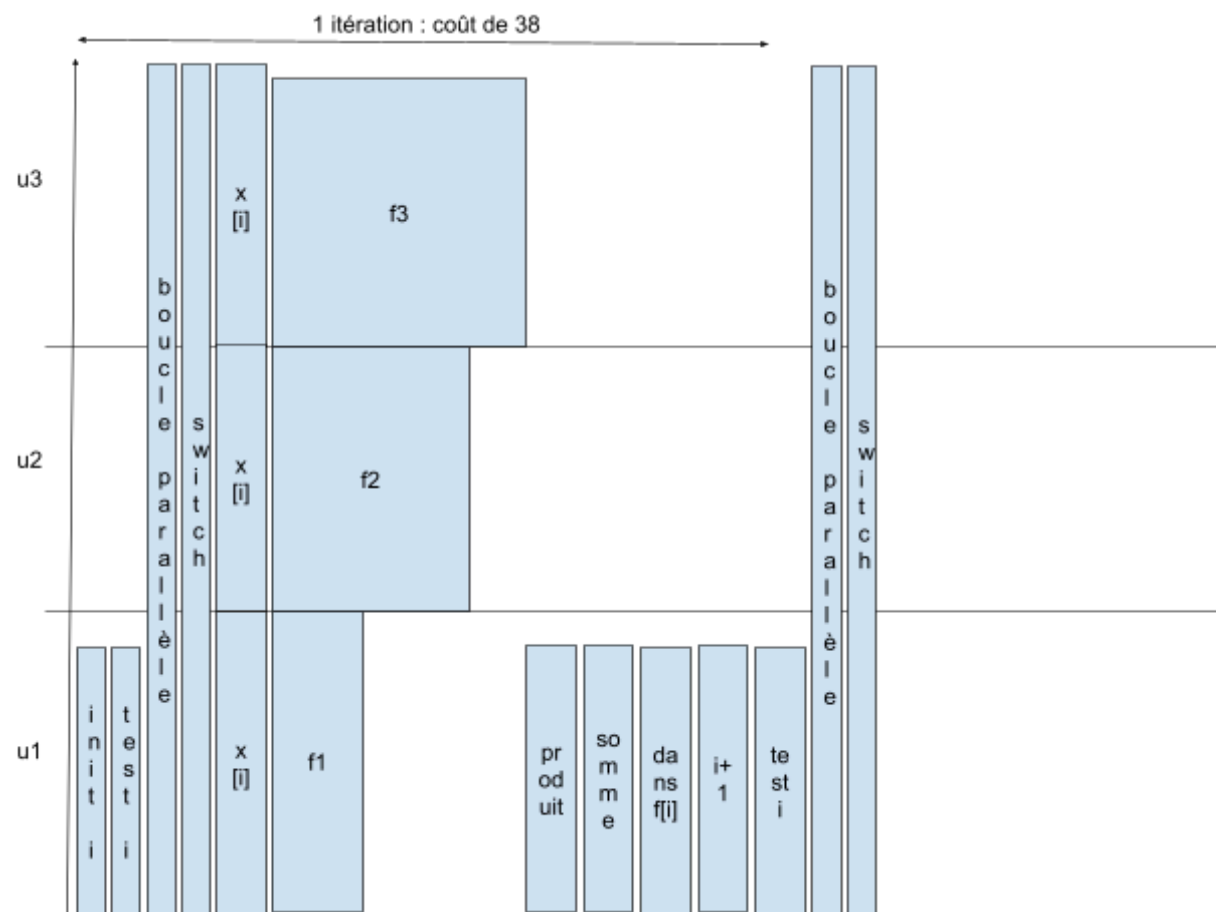
Maintenant on fait un algo en parallélisme de tâches avec 3 unités

- activation des unités, procédure de mise en place de boucle parallèle = coût de 1

```

pour i de 0 à N-1 faire
    pour u de 1 à 3 faire en parallèle
        switch u :
            case 1: res1 <- f1(x[i])
            case 2 : res2 <- f2(x[i])
            case 3 : res3 <- f3(x[i])
        finswitch
    finpour
    f[i] = res1*res2+res3
finpour
    
```

Diagramme temporel :



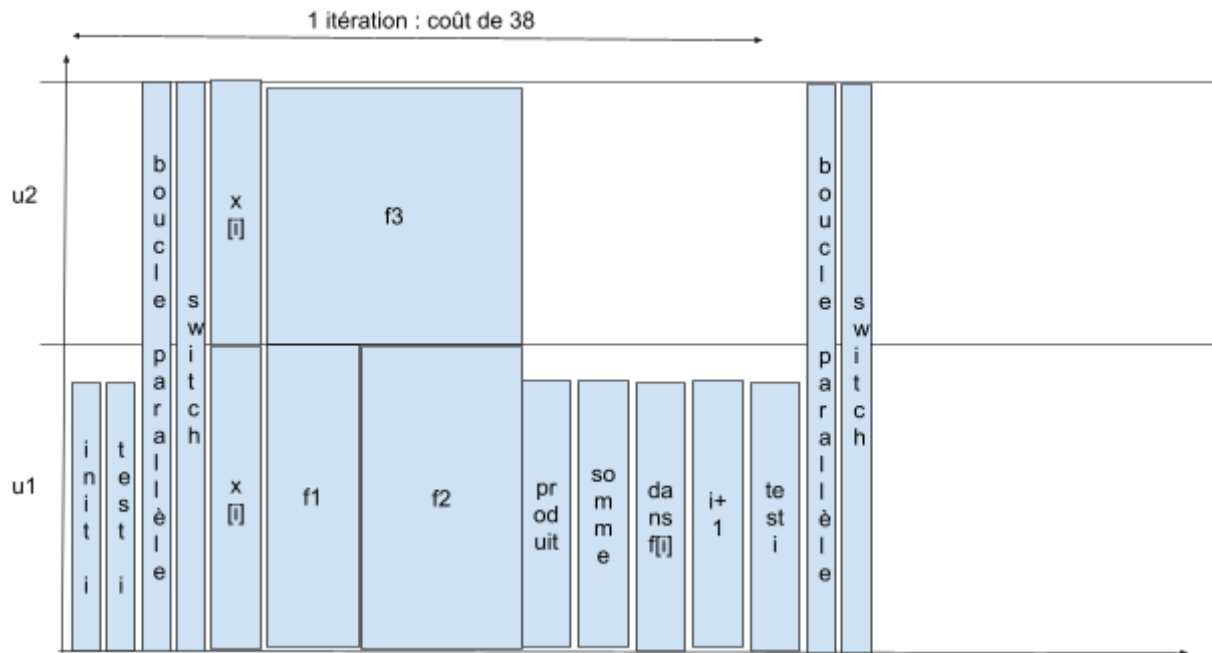
$$T3 = 1 + 38n$$

$$S3 = 1 + 66n / 1 + 38n \approx 66/38 = 1,74$$

$$E3 = 1,74/3 = 0,58$$

Efficacité pas folle, y a pas mal de trous

Cependant, on peut réduire pour avoir deux unités seulement. Regardez :



```

pour i de 0 à N-1 faire
  pour u de 1 à 3 faire en parallèle
    switch u :
      case 1: res1 <- f1(x[i]) * f2(x[i])
      case 2 : res2 <- f3(x[i])
    finswitch
  finpour
  f[i] = res1+res2
finpour

```

$$T2 = 1 + 38n$$

$$S2 = 1+66n / 1+38n \approx 66/38 = 1,74$$

$$E2 = 1,74/2 = 0,87$$

Now we're talking 😎